



UNIVERSIDAD
CATÓLICA
BOLIVIANA

INFORME PRIMERA EVALUACIÓN

ASIGNATURA INTERNET DE LAS COSAS

**GRUPO N.º
INTEGRANTES**

-
1. Luis Andres Menchaca Rivero
 2. Yudith Noa Vargas

**DOCENTE
CARRERA
FECHA DE ENTREGA
GESTIÓN**

Ing. Pamela Valenzuela
Ingeniería de Sistemas
10 de septiembre de 2026
II/2026

**LA PAZ - BOLIVIA
2026**

1. Introducción

Descripción del problema y solución propuesta

El proyecto resuelve el problema de aproximar funciones matemáticas utilizando series numéricas. El usuario puede seleccionar una serie, ingresar el valor de (x) y definir el número de términos (n). El sistema calcula:

- El valor aproximado mediante una serie matemática.
- El valor real utilizando funciones de Python.
- El error absoluto.
- El error relativo.
- El registro asociado al usuario.

La solución utiliza una arquitectura cliente-servidor. El cliente puede ser una interfaz web o un cliente de consola. El servidor, desarrollado con FastAPI, procesa las solicitudes, ejecuta los cálculos y almacena los resultados en una base de datos SQLite.

A. Series matemáticas seleccionadas

Serie de Taylor de la Trigonometría Directa

Las funciones trigonométricas directas pueden aproximarse mediante sus expansiones en serie de Taylor alrededor de cero:

- Seno:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

- Coseno:

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

- Tangente:

$$\tan(x) = \frac{\sin(x)}{\cos(x)}$$

Características matemáticas: Son series infinitas alternantes que utilizan potencias impares o pares de la variable . El factorial en el denominador favorece la rápida convergencia.

Aplicaciones: Cálculo numérico en computación científica, simulación de movimientos periódicos y procesamiento de señales.

B. Serie de Taylor de la Trigonometría Inversa

- Serie de Taylor del arcoseno

La función trigonométrica inversa se representa mediante la expansión:

$$\arcsin(x) = x = \left(\frac{1}{2}\right)\left(\frac{x^3}{3}\right) + \frac{(1 \cdot 3)}{(2 \cdot 4)}\left(\frac{x^5}{5}\right) + \dots$$

Características matemáticas: Requiere validación del dominio. Converge bien para valores en $[-1, 1]$.

- Serie de Taylor del arcocoseno

$$\arccos(x) = \frac{\pi}{2} - \arcsin(x)$$

La función trigonométrica inversa se representa mediante la expansión:

Características matemáticas: Esto evita la necesidad de una serie separada y aprovecha la relación complementaria entre ambas funciones inversas.

- Serie de Taylor del arcotangente

La función trigonométrica inversa se representa mediante la expansión:

$$\arctan(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots$$

Características matemáticas: Serie alternante con potencias impares. Converge de forma confiable en el intervalo $[-1, 1]$.

Aplicaciones: Cálculo de funciones trigonométricas inversas, métodos numéricos y desarrollo de modelos matemáticos avanzados.

Utilidad de las series

Las series permiten estudiar métodos de aproximación numérica. Son útiles para:

- Comprender cómo funcionan las funciones matemáticas internamente.
- Comparar valores aproximados y valores reales.
- Analizar el efecto del número de términos.

- Medir la precisión mediante el error absoluto y relativo.
- Aplicar métodos numéricos en sistemas cliente-servidor.

Interacción entre componentes

El flujo general es:

1. El usuario ingresa su nombre, la serie, el valor de (x) y (n) .
2. El cliente envía una solicitud HTTP al servidor.
3. FastAPI recibe y valida los datos.
4. El servidor calcula la aproximación y el valor real.
5. Se calculan los errores.
6. El resultado se almacena en SQLite.
7. El servidor devuelve una respuesta JSON.
8. El dashboard consulta el historial y genera estadísticas y gráficos.

2. Objetivos

2.1 Objetivo General

La base de datos del proyecto fue implementada utilizando SQLite mediante SQLAlchemy ORM, un sistema de base de datos relacional que estructura la información en tablas con relaciones explícitas entre sí. Esta arquitectura permite almacenar de manera consistente y estructurada tanto la información de los usuarios como los registros correspondientes a los cálculos de las series de Taylor.

2.2 Objetivos Específicos

- Implementar la comunicación entre un cliente y un servidor mediante HTTP.
- Validar los datos enviados por el usuario.
- Implementar series matemáticas para seno, coseno y tangente.
- Comparar los valores aproximados con los valores reales.
- Calcular el error absoluto y relativo.
- Registrar los cálculos realizados por cada usuario.
- Utilizar una base de datos relacional para almacenar la información.
- Crear un dashboard con estadísticas, gráficos e historial.
- Manejar errores matemáticos, datos inválidos y usuarios inexistentes.
- Verificar la precisión de las series mediante pruebas unitarias.

3. Diseño de la solución cliente-servidor

Arquitectura

La aplicación está organizada en los siguientes componentes:

Cliente web

Se encuentra principalmente en `templates/index.html`. Permite ingresar:

- Nombre del usuario.
- Tipo de serie.
- Valor de (x) .
- Número de términos (n) .

El cliente envía los datos mediante `fetch()` al endpoint:

```
POST /calcular
```

Cliente de consola

Está implementado en `cliente/cliente.py`. Se ejecuta mediante:

```
python -m cliente.cliente
```

Permite seleccionar la serie, ingresar los datos y enviar la solicitud al servidor.

Servidor

El servidor utiliza FastAPI y se inicializa en `main.py`.

Sus responsabilidades son:

- Recibir solicitudes HTTP.
- Validar los datos.
- Ejecutar las series matemáticas.
- Calcular los errores.
- Guardar los resultados.
- Responder al cliente.

La lógica principal de cálculo está en `routers/calculo.py`.

Base de datos

La aplicación utiliza SQLite mediante SQLAlchemy. La configuración está en [config.py](#):

```
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./series.db")
```

Dashboard

El dashboard se encuentra en templates/dashboard.html. Consulta los datos mediante:

```
GET /api/historial/{nombre_usuario}
```

y utiliza Chart.js para mostrar gráficos.

Flujo de datos

```
Usuario
|
v
Cliente web o cliente de consola
|
| POST /calcular
v
Servidor FastAPI
|
+--> Valida datos
|
+--> Ejecuta serie matemática
|
+--> Calcula valor real y errores
|
+--> Guarda registro en SQLite
|
v
Respuesta JSON
|
v
Dashboard e historial
```

Identificación del usuario

El usuario se identifica mediante el campo `nombre_usuario`.

Si el usuario no existe, el servidor lo crea automáticamente mediante la función:

```
obtener_o_crear_usuario()
```

Esta función está en `routers/calculo.py`.

Actualmente no existe autenticación con contraseña. El nombre de usuario funciona como identificador lógico.

Número de registro (n)

Existen dos conceptos relacionados:

`n_terminos`: número de términos usados para aproximar la serie.

`n_registro`: identificador automático de cada cálculo almacenado.

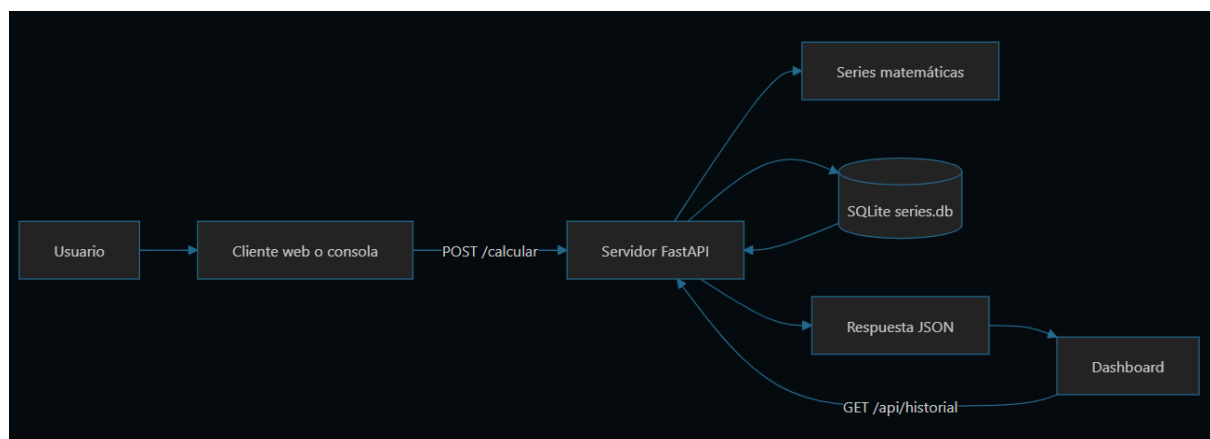
Por ejemplo:

```
n_terminos = 15
```

```
n_registro = 8
```

Esto significa que el cálculo número 8 fue realizado utilizando 15 términos de la serie.

Diagrama de arquitectura



4. Diseño de la Base de Datos SQL

Nombre y tecnología

- Base de datos: `series.db`
- Tecnología: `SQLite`
- ORM: `SQLAlchemy`
- Archivo de conexión: `database/db.py`

Estructura de tablas

Tabla usuarios: Almacena la información de los operadores registrados en el sistema (identificador único, nombre de usuario y fecha de registro).

Tabla tipos_serie: Almacena el catálogo de tipos de series soportados por el sistema, clasificándolos por nombre (por ejemplo, Taylor seno o Trigonométrica tangente) y categoría (Taylor, Trigonometrica o Trig_inversa).

Tabla registros_calculo: Almacena los resultados de las aproximaciones calculadas, registrando campos como identificador de usuario, identificador del tipo de serie, valor de entrada, número de términos, resultado aproximado, valor real, error absoluto, error relativo y marca de tiempo. Se relaciona con usuarios y tipos_serie mediante claves foráneas.

Relaciones

usuarios 1 ————— N registros_calculo

tipos_serie 1 ————— N registros_calculo

Cada registro pertenece a un usuario y a un tipo de serie.

Restricciones

- Las claves primarias utilizan primary_key=True.
- Los identificadores se generan automáticamente.
- nombre_usuario es único y obligatorio.
- Los campos principales de los cálculos son NOT NULL.
- id_usuario y id_tipo son claves foráneas.
- n_terminos debe estar entre 1 y 200.

La validación del rango se encuentra en database/[schemas.py](#):

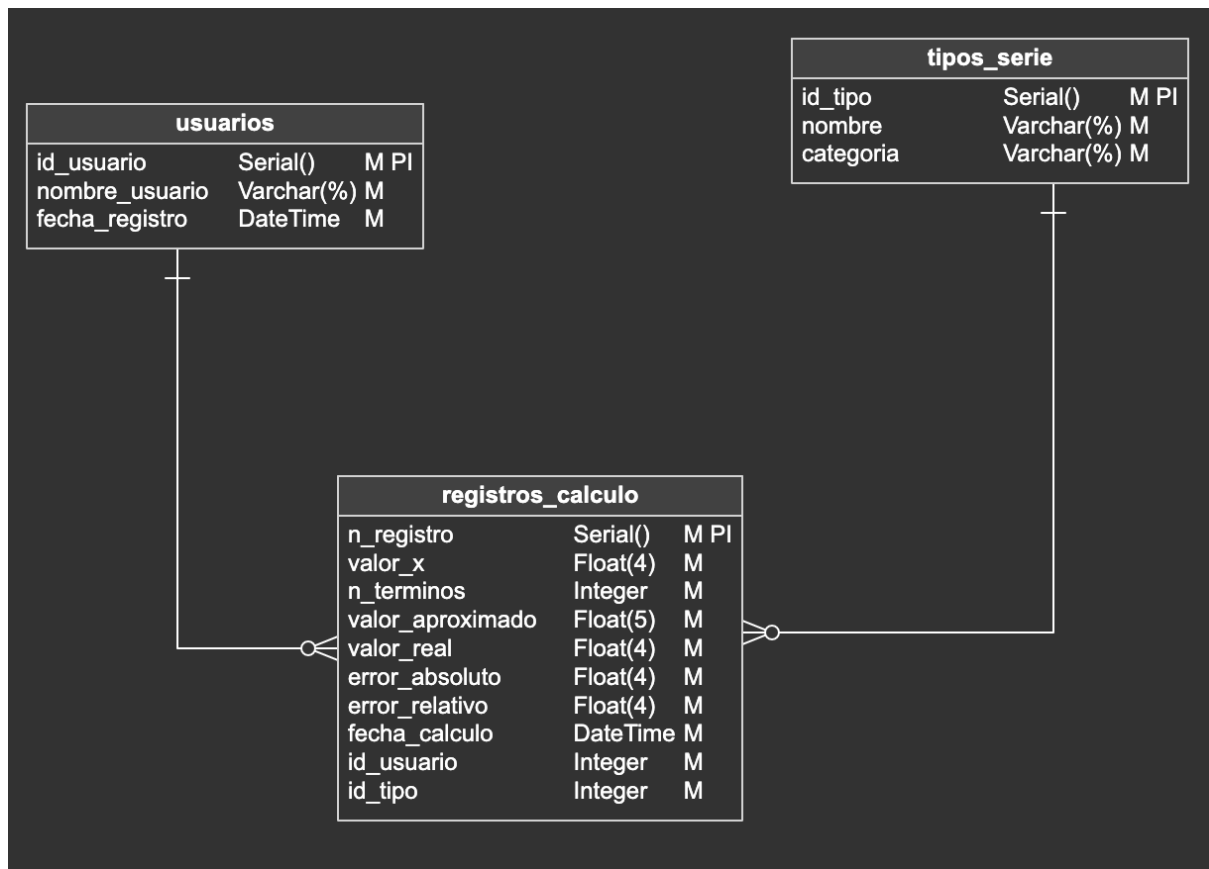
```
n_terminos: int = Field(gt=0, le=200)
```

Justificación de los tipos

- Integer: identificadores y cantidades de términos.
- Float: valores matemáticos y errores.
- String: nombres y categorías.
- DateTime: fecha y hora del cálculo.

- Claves foráneas: evitan repetir información de usuarios y tipos de serie.

Diagrama entidad-relación



5. Desarrollo de la Aplicación

Tecnologías utilizadas

- Python
- FastAPI
- Uvicorn
- SQLAlchemy
- SQLite
- Pydantic
- Jinja2
- JavaScript
- Chart.js
- Pytest
- Las dependencias se encuentran en requirements.txt.

Implementación del cliente

El cliente web utiliza un formulario HTML y JavaScript. La función `calcular()` construye un objeto JSON:

```
const payload = {  
  nombre_usuario: form.nombre_usuario.value.trim(),  
  tipo_serie: form.tipo_serie.value,  
  valor_x: parseFloat(form.valor_x.value),  
  n_terminos: parseInt(form.n_terminos.value),  
};
```

Después envía la solicitud:

```
fetch('/calcular', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(payload),  
});
```

Código disponible en `templates/index.html`.

El cliente de consola realiza una función equivalente utilizando `urllib.request`.

Implementación del servidor

El endpoint principal es:

```
@router.post("/calcular", response_model=CalculoResponse)  
def calcular_serie(datos: CalculoRequest, db: Session =  
Depends(get_db)):
```

El servidor:

- Recibe la solicitud.
- Obtiene la función matemática correspondiente.
- Calcula la aproximación.
- Calcula el valor real.
- Calcula los errores.
- Crea o busca el usuario.
- Inserta el registro.
- Devuelve el resultado.

Implementación de las series

El diccionario FUNCIONES_SERIE relaciona cada tipo de serie con:

La función aproximada.

La función real de Python.

```
FUNCIONES_SERIE = {  
    TipoSerieEnum.taylor_seno: (taylor_seno, math.sin),  
    TipoSerieEnum.taylor_coseno: (taylor_coseno, math.cos),  
    TipoSerieEnum.trig_tangente: (trig_tangente, math.tan),  
}
```

Cálculo del error

El error absoluto se calcula así:

```
error_absoluto = abs(valor_real - valor_aproximado)
```

El error relativo se calcula mediante:

```
error_relativo = (  
    error_absoluto / abs(valor_real)  
    if valor_real != 0  
    else None  
)
```

Cuando el valor real es cero, el error relativo se guarda como NULL porque no es posible dividir entre cero.

Inserción en la base de datos

El registro se crea mediante:

```
registro = RegistroCalculo(  
    id_usuario=usuario.id_usuario,  
    id_tipo=tipo_serie.id_tipo,  
    valor_x=datos.valor_x,  
    n_terminos=datos.n_terminos,  
    valor_aproximado=valor_aproximado,  
    valor_real=valor_real,  
    error_absoluto=error_absoluto,  
    error_relativo=error_relativo,  
)  
db.add(registro)
```

```
db.commit()
db.refresh(registro)
```

La conexión y las sesiones se administran en database/db.py.

Lectura de datos

El dashboard consulta los registros mediante:

```
@router.get("/api/historial/{nombre_usuario}")
def historial_detalle(nombre_usuario: str, db: Session = Depends(get_db)):
```

Los registros se filtran por usuario, se ordenan por n_registro y se devuelven como JSON.

Otras funciones

- Validación de valores mediante Pydantic.
- Creación automática de usuarios.
- Catálogo inicial de series.
- Manejo de dominios inválidos en arcoseno y arcotangente.
- Control de errores cuando la tangente no está definida.
- Documentación automática de la API en /docs.

6. Desarrollo del Dashboard en Python para Visualización en Tiempo Real

Diseño y herramientas

El dashboard utiliza:

- HTML y Jinja2 para la estructura.
- JavaScript para consultar la API.
- Chart.js para los gráficos.
- CSS personalizado para el diseño visual.

El dashboard se encuentra en templates/dashboard.html.

Estadísticas

Se muestran:

- Total de cálculos.

- Error promedio.
- Serie más utilizada.
- Fecha del último cálculo.

El error promedio se calcula en JavaScript:

```
const avgError =

    data.reduce((s, r) => s + r.error_absoluto, 0)

    / (data.length || 1);
```

Gráficos

El sistema muestra tres gráficos principales:

- Valor aproximado contra valor real. Permite comparar visualmente la precisión.
- Error absoluto por registro. Muestra el error de cada cálculo.
- Distribución por tipo de serie. Indica cuántas veces fue utilizada cada serie.

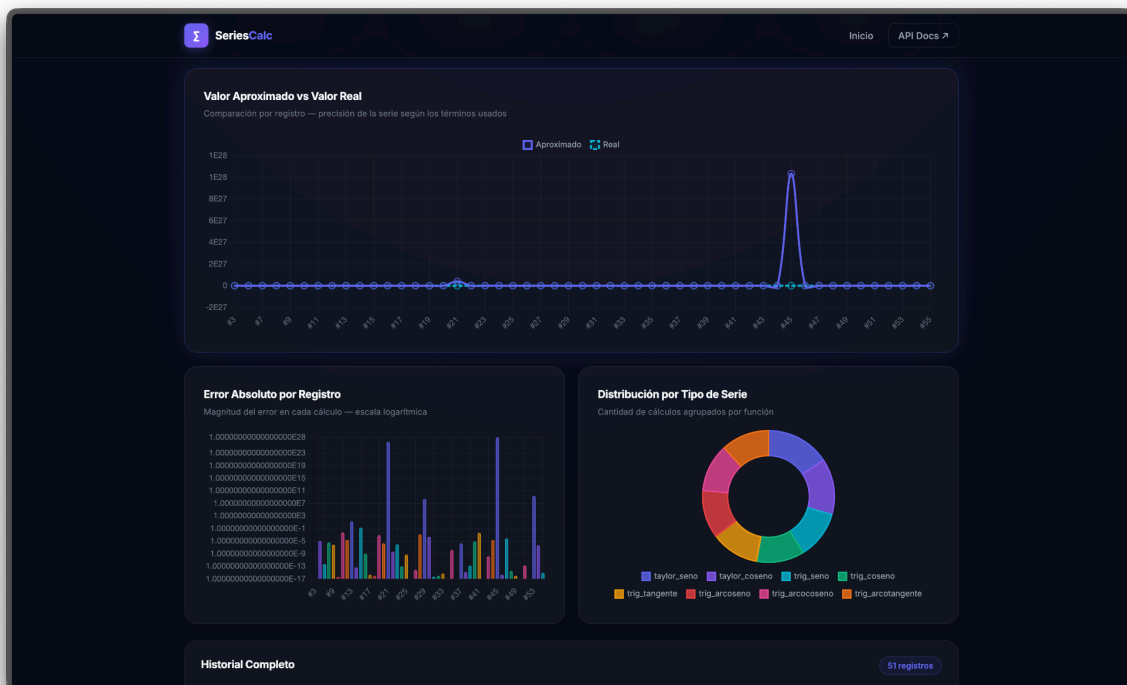


Tabla de historial

La tabla contiene:

- Número de registro.
- Tipo de serie.
- Categoría.
- Valor de (x).
- Número de términos.
- Valor aproximado.
- Valor real.
- Error absoluto.
- Error relativo.
- Fecha del cálculo.

Actualización de datos

Actualmente el dashboard realiza una consulta cuando se carga la página:

```
fetch(`/api/historial/${encodeURIComponent(USUARIO)}`)
```

Interactividad

La aplicación permite:

- Seleccionar el tipo de serie.
- Elegir el usuario.
- Ingresar (x) y (n).
- Consultar el historial.
- Visualizar gráficos dinámicos.
- Abrir el dashboard desde el resultado del cálculo.

7. Pruebas y Resultados

Comunicación cliente-servidor

Se debe demostrar que el cliente puede enviar una solicitud al servidor y recibir una respuesta.

Ejemplo de solicitud:

```
{  
  
  "nombre_usuario": "ana",  
  
  "tipo_serie": "taylor_seno",  
  
  "valor_x": 0.785398,  
  
  "n_terminos": 15  
  
}
```

Ejemplo de respuesta:

```
{  
  
  "n_registro": 1,  
  
  "valor_aproximado": 0.707106,  
  
  "valor_real": 0.707106,  
  
  "error_absoluto": 0.000000,  
  
  "error_relativo": 0.000000,  
  
  "fecha_calculo": "2026-09-10T20:00:00"  
  
}
```

Generación de las tres series

Las pruebas unitarias están en tests/test_series.py.

Se prueban:

- Taylor seno.
- Taylor coseno.
- Seno y coseno trigonométricos.
- Tangente.
- Arcoseno.

- Arcocoseno.
- Arcotangente.
- Valores fuera del dominio.
- Casos cercanos a puntos problemáticos.

Se ejecuta:

```
pytest tests/test_series.py
```

```

> source venv/bin/activate && pytest -v
===== test session starts =====
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Users/andres/Documents/locker-go/venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/andres/Documents/locker-go
plugins: anyio-4.12.1
collected 35 items

tests/test_series.py::TestTaylorSeno::test_aproximacion[0] PASSED [ 2%]
tests/test_series.py::TestTaylorSeno::test_aproximacion[0.5235987755982988] PASSED [ 5%]
tests/test_series.py::TestTaylorSeno::test_aproximacion[0.7853981633974483] PASSED [ 8%]
tests/test_series.py::TestTaylorSeno::test_aproximacion[1.5707963267948966] PASSED [ 11%]
tests/test_series.py::TestTaylorSeno::test_aproximacion[3.141592653589793] PASSED [ 14%]
tests/test_series.py::TestTaylorSeno::test_un_termino_es_x PASSED [ 17%]
tests/test_series.py::TestTaylorCoseno::test_aproximacion[0] PASSED [ 20%]
tests/test_series.py::TestTaylorCoseno::test_aproximacion[0.5235987755982988] PASSED [ 22%]
tests/test_series.py::TestTaylorCoseno::test_aproximacion[0.7853981633974483] PASSED [ 25%]
tests/test_series.py::TestTaylorCoseno::test_aproximacion[1.5707963267948966] PASSED [ 28%]
tests/test_series.py::TestTaylorCoseno::test_aproximacion[3.141592653589793] PASSED [ 31%]
tests/test_series.py::TestTaylorCoseno::test_un_termino_es_uno PASSED [ 34%]
tests/test_series.py::TestTrigonometricas::test_seno_delega_en_taylor PASSED [ 37%]
tests/test_series.py::TestTrigonometricas::test_coseno_delega_en_taylor PASSED [ 40%]
tests/test_series.py::TestTrigonometricas::test_tangente[0] PASSED [ 42%]
tests/test_series.py::TestTrigonometricas::test_tangente[0.5] PASSED [ 45%]
tests/test_series.py::TestTrigonometricas::test_tangente[1.0] PASSED [ 48%]
tests/test_series.py::TestTrigonometricas::test_tangente_indefinida_lanza_error PASSED [ 51%]
tests/test_series.py::TestTrigInversas::test_arccoseno[-0.9-50-0.001] PASSED [ 54%]
tests/test_series.py::TestTrigInversas::test_arccoseno[-0.5-15-1e-06] PASSED [ 57%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0-15-1e-06] PASSED [ 60%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0.5-15-1e-06] PASSED [ 62%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0.9-50-0.001] PASSED [ 65%]
tests/test_series.py::TestTrigInversas::test_arccoseno_fuera_dominio PASSED [ 68%]
tests/test_series.py::TestTrigInversas::test_arccoseno[-0.9-50-0.001] PASSED [ 71%]
tests/test_series.py::TestTrigInversas::test_arccoseno[-0.5-15-1e-06] PASSED [ 74%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0-15-1e-06] PASSED [ 77%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0.5-15-1e-06] PASSED [ 80%]
tests/test_series.py::TestTrigInversas::test_arccoseno[0.9-50-0.001] PASSED [ 82%]
tests/test_series.py::TestTrigInversas::test_arcotangente[-0.9-50-0.001] PASSED [ 85%]
tests/test_series.py::TestTrigInversas::test_arcotangente[-0.5-15-1e-06] PASSED [ 88%]
tests/test_series.py::TestTrigInversas::test_arcotangente[0-15-1e-06] PASSED [ 91%]
tests/test_series.py::TestTrigInversas::test_arcotangente[0.5-15-1e-06] PASSED [ 94%]
tests/test_series.py::TestTrigInversas::test_arcotangente[0.9-50-0.001] PASSED [ 97%]
tests/test_series.py::TestTrigInversas::test_arcotangente_fuera_dominio PASSED [100%]

===== 35 passed in 0.04s =====

```

Registros asociados a usuarios y valores (n)

Se deben realizar cálculos con usuarios diferentes y verificar que:

- Cada usuario tenga sus propios registros.

- Cada cálculo tenga un n_registro.
- Cada registro conserve el tipo de serie.
- El campo n_terminos almacene el número de términos utilizado.

```

> sqlite3 -header -column series_db "SELECT r.n_registro, u.nombre_usuario, t.nombre AS serie, r.valor_x, r.n_terminos, ROUND(r.valor_aproximado, 6) AS aprox, ROUND(r.valor_real, 6) AS real, r.error_absoluto FROM registros_calculo r JOIN usuarios u ON r.id_usuario = u.id_usuario JOIN tipos_serie t ON r.id_tipo = t.id_tipo LIMIT 15;"
n_registro  nombre_usuario  serie          valor_x      n_terminos  aprox        real          error_absoluto
-----
1 demo        taylor_seno     0.785         10           0.786825    0.786825     0.0
2 test        taylor_seno     0.5           10           0.479426    0.479426     0.0
3 andres      taylor_seno     0.785         100          0.786825    0.786825     0.0
4 axl         trig_coseno     0.8           200          0.696707    0.696707     0.0
5 axl         taylor_coseno   0.9           200          0.62161     0.62161      1.1102238246251565e-16
6 andres      taylor_seno     1.7528888229953293  5           0.983623    0.983611     1.1734273354768732e-05
7 andres      taylor_coseno   3.8354131020657356  12          -0.994368    -0.994368     5.943823858818463e-13
8 andres      trig_seno       -3.4782364464968003  8           0.338325    0.338321     4.3335385382434304e-06
9 andres      trig_coseno     2.2204712463236814  7           -0.684927    -0.684928     7.9607451419771991e-07
10 andres     trig_tangente    0.27147753734711899  6           0.278349    0.278349     5.5511151231257827e-17
11 andres     trig_coseno     -0.93108150551262073  11          -1.190606    -1.197147     0.0065406151531857137
12 andres     trig_arccoseno  -0.52933143108633562  5           2.128582    2.128609     2.620247767515238e-05
13 andres     trig_arcotangente 1.2249012587722596  18          -19.415171    0.88614      20.301310193299045
14 andres     taylor_seno     -3.5138467521563182  13          0.362971    0.362971     4.8350212722425567e-14
15 andres     taylor_coseno   3.8884177979839798  5           -0.53909    -0.733849     0.19475940170872041

```

Análisis de resultados

Al aumentar el número de términos, normalmente la aproximación se acerca más al valor real. Esto provoca una reducción del error absoluto y relativo.

Sin embargo, algunas series, como arcsin y arctan, convergen lentamente cuando (x) se aproxima a 1 o -1. Por esta razón, el proyecto utiliza tolerancias diferentes en las pruebas.

La tangente presenta una condición especial: cuando el coseno es cercano a cero, la división se vuelve inestable y el sistema genera un error controlado.

8. Conclusiones

El proyecto cumple con el objetivo de implementar una aplicación cliente-servidor para el cálculo y análisis de series matemáticas.

La solución permite:

- Calcular aproximaciones de funciones trigonométricas.
- Comparar resultados aproximados y reales.
- Calcular errores numéricos.
- Asociar registros con usuarios.
- Persistir la información en SQLite.
- Consultar el historial mediante una API.
- Visualizar los datos mediante un dashboard.

La arquitectura separa correctamente el cliente, servidor, lógica matemática y persistencia. Además, el uso de FastAPI facilita la validación, documentación y exposición de los servicios.

Como posibles mejoras futuras se pueden agregar:

- Autenticación real de usuarios.
- Actualización automática del dashboard.
- Filtros por rango de fechas o tipo de serie.
- Exportación de resultados a CSV.
- Migración de SQLite a PostgreSQL para producción.
- Gráficos adicionales relacionados con (n) y el error.

7. Anexos

Repositorio del Proyecto: <https://github.com/menchaca-andres/locker-go>

Estructura de Código FastAPI:

```
# routes/trigonometria.py

@router.get("/api/historial/{nombre_usuario}",
            response_model=List[RegistroDetalle])

def historial_detalle(nombre_usuario: str,
                      db: Session = Depends(get_db)):

    usuario = db.query(Usuario) \
        .filter_by(nombre_usuario=nombre_usuario).first()

    if not usuario:

        raise HTTPException(status_code=404,
                             detail="Usuario no encontrado")

    registros = db.query(RegistroCalculo) \
        .filter_by(id_usuario=usuario.id_usuario) \
        .order_by(RegistroCalculo.n_registro).all()
```

```

return [

RegistroDetalle(

    n_registro=r.n_registro,

    tipo_serie=r.tipo_serie.nombre,

    categoria=r.tipo_serie.categoria,

    valor_x=r.valor_x,

    n_terminos=r.n_terminos,

    valor_aproximado=r.valor_aproximado,

    valor_real=r.valor_real,

    error_absoluto=r.error_absoluto,

    error_relativo=r.error_relativo,

    fecha_calculo=r.fecha_calculo,

)

for r in registros

]

```

Modelos SQLAlchemy (SQLite):

```

# database/models.py

class Usuario(Base):

    __tablename__ = "usuarios"

    id_usuario = Column(Integer, primary_key=True)

    nombre_usuario = Column(String, unique=True)

    password = Column(String)

    fecha_registro = Column(DateTime, default=datetime.now)


class TipoSerie(Base):

    __tablename__ = "tipo_serie"

    id_tipo = Column(Integer, primary_key=True)

    nombre = Column(String) # seno, coseno, etc.

```

```
categoria = Column(String) # Directa o Inversa
```

```
class RegistroCalculo(Base):
```

```
    __tablename__ = "registro_calculo"
```

```
    n_registro = Column(Integer, primary_key=True)
```

```
    id_usuario = Column(Integer, ForeignKey("usuarios.id_usuario"))
```

```
    id_tipo_serie = Column(Integer, ForeignKey("tipo_serie.id_tipo"))
```

```
    valor_x = Column(Float)
```

```
    n_terminos = Column(Integer)
```

```
    valor_aproximado = Column(Float)
```

```
    valor_real = Column(Float)
```

```
    error_absoluto = Column(Float)
```

```
    error_relativo = Column(Float)
```

```
    fecha_calculo = Column(DateTime, default=datetime.now)
```